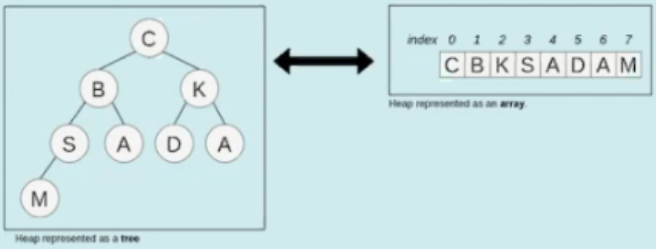
Max Heap

a) Create a **MaxHeap**, using the **bottom-up** construction approach, from the given sequence of integer numbers [13, 19, 7, 28, 12, 34, 21, 15, 2, 38, 25, 5, 16, 18, 10] and **write it down** in an **array** form in the table below.

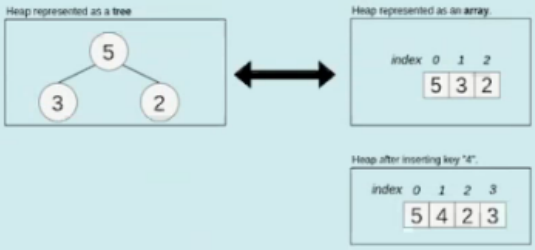
*The graphical example below shows the conversion of a binary tree into an array. Index position 0 represents the root, the children of a node with index i have the indices 2*i+1 (left child) and 2*i+2 (right child).*



b) Now insert a node with **key 37** into the following MaxHeap, **represented in form of the following array:**

index	0	1	2	3	4	5	6	7	8	9
	29	21	18	7	19	15	1	3	6	12

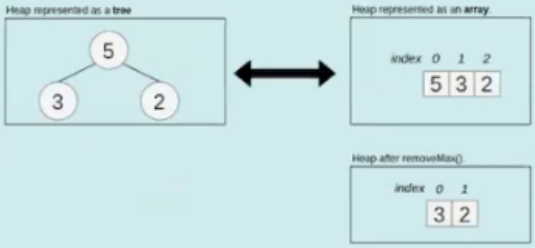
The graphic example below shows a MaxHeap before and after inserting a key.



c) Call the method **removeMax()** on the following **MaxHeap**:

index	0	1	2	3	4	5	6	7	8	9	10
	39	35	25	19	28	21	13	11	17	5	1

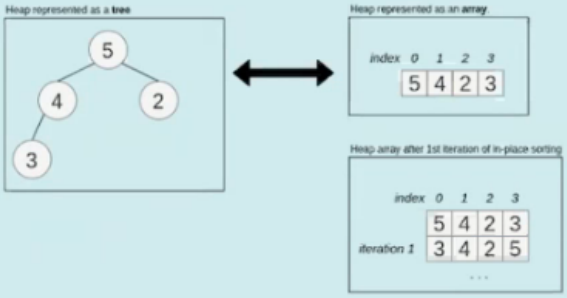
The graphic example below shows a MaxHeap before an after calling removeMax().



d) Perform the **in-place** sort on the following **MaxHeap** array and write down the content of the array after each iteration to show the intermediate steps:

index	0	1	2	3	4	5	6	7	8	9	10
	39	35	25	19	28	21	13	11	17	5	1

The graphic example below shows the first iteration of the in-place sorting of a MaxHeap:



Binary Search Tree

a) Use the following **sequence** of keys to create a valid **Binary Search Tree**: **17, 10, 29, 1, 8, 13, 16, 18, 25**

Drag&Drop the keys (according to the sequence order, 17 in the root and so on) below the box into the tree skeleton.

Note: Each key can be used only once and therefore some edges of the skeleton will remain unused.

b) **Use** your Binary Search Tree created in **example a)** and remove the root (17). Update the remaining Binary Search Tree by using the **inorder successor** of 17 (again using Drag&Drop).

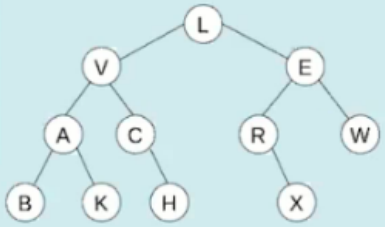
Note: Each key can be used only once and therefore some edges of the skeleton will remain unused.

Create a **Binary Search Tree** using Drag&Drop with the nodes provided (not necessarily in the given order), that represents the **worst thinkable case** of a **degenerated binary search tree**.

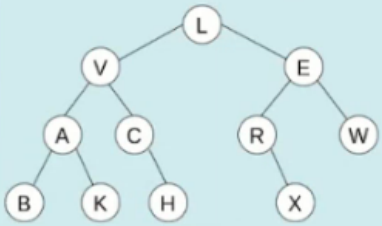
Note: Each key can be used only once and therefore some edges of the skeleton will remain unused. **[7, 21, 3, 31, 15]**

Tree Traversal

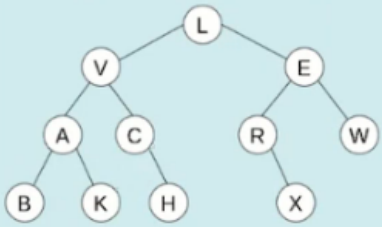
Write down the **node sequence** for the **inorder traversal** of the following tree, e.g.: A,B,C,...



Write down the **node sequence** for the **preorder traversal** of the following tree, e.g.: A,B,C,...



Write down the **node sequence** for the **postorder traversal** of the following tree, e.g.: A,B,C,...



Sorting

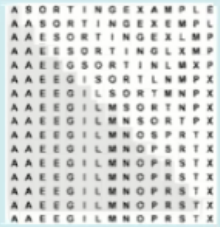
Complete the following implementation of the Radix-Exchange-Sort algorithm, which sorts positive integer numbers in the range of 0 to ($2^{numOfBits}$-1). You can assume, that there is a method **bitAt()** which supports you to extract each single bit of a number (least significant bit is at position 0).

The algorithm starts with the most significant bit (position: **numOfBits-1**) and places all numbers with bit 0 at this position in the front part of the list and all numbers with bit 1 in the rear part of the list. For that it uses two indexes i and j to search in opposite directions for two numbers in the wrong part of the list and swaps them. Then the next bit position is examined and the algorithm for the two resulting sublists is called recursively.

```
class Radix:

    def __init__(self):
        self.nBits = 16

    def bitAt(self, value, index):
    # E.g.: value 15413 (0011110000110101): index 0 -> Bit 1, index 1 -> Bit 0, ...
        return value & 1 << index != 0

    def radix_sort(self, array):
        return self.sort(array, self.nBits - 1, 0, len(array) - 1)

    def sort(self, array, bitPos, left, right):
        if ((bitPos >= 0) and (left < right)):   # termination condition
            i = left   # i searches from left to right for a number with Bit 1 at bitPos
            j = right  # j searches from right to left for a number with Bit 0 at bitPos

            while (i <= j): # find the bit positions to be swapped
            < INSERT YOUR CODE PART 1 HERE >          # search for 0 from left

            < INSERT YOUR CODE PART 2 HERE >          # search for 1 from right

                if (i < j):
                    h = array[i];  array[i] = array[j];  array[j] = h;       # swap

            array = self.sort(array,bitPos-1, left, j)  # continue with next bit
            array = self.sort(array,bitPos-1, i, right) # continue with next bit
        return array
```

Sort the array [9,12,23,7,21,2,17,6] in ascending order using the **BubbleSort** algorithm and write down the content of the array **after each bubble iteration** in the table below.

The example below shows the BubbleSort algorithm:



Sort the array **[9,12,23,7,21,2,17,6]** in ascending order using the **InsertionSort** algorithm and write down the content of the array **after each insertion step** in the table below.

The example below shows the InsertionSort algorithm.

Sort the array **[9,12,23,7,21,2,17,6]** using the **MergeSort** algorithm. Write down the (intermediate) result **after each merge step** in the table below.

The example below shows the MergeSort algorithm.